Цель: Закрепить понимание инкапсуляции, полиморфизма, перегрузки операторов и наследования на практике.



Задача 1. Инкапсуляция: «Умный кошелёк»

Создайте класс `Wallet`.

1. Конструктор `__init__(self, initial_balance)` принимает начальный баланс. Если передано отрицательное число, баланс должен стать `0`.
2. Метод `deposit(amount)` пополняет счёт. Если `amount <= 0`, метод ничего не делает и возвращает `False`, иначе возвращает `True`.

3. Метод `withdraw(amount)` снимает средства. Если `amount <= 0` или средств не хватает, возвращает `False`, иначе возвращает `True`.
4. Метод `get_balance()` возвращает текущий баланс. Численное значение

Примечание: Атрибут баланса должен быть защищённым (соглашение об инкапсуляции).

Задача 2. Полиморфизм: «Склад фигур»

Реализуйте классы `Triangle` (принимает 3 стороны: a, b, c) и `Circle` (принимает радиус r). Оба класса должны иметь метод `perimeter()`.

Напишите полиморфную функцию `calculate_total_perimeter(shapes)`, которая принимает список любых фигур и возвращает сумму их периметров. Функция не должна использовать `isinstance` или `type`. Функция должна работать с любыми объектами, у которых есть метод `perimeter()`

Создайте класс `Vector2D` с атрибутами `x` и `y`.

Переопределите следующие операторы:

1. `+` (`__add__`): возвращает новый вектор, являющийся суммой двух векторов.
2. `*` (`__mul__`): если второй операнд — число, возвращает новый вектор, умноженный на это число (скалярное умножение).
3. `==` (`__eq__`): векторы равны, если равны их `x` и `y`.
4. `__str__`: возвращает строку вида `"(x, y)"` (например, `"(3, 4)"`).
5. `__hash__`: векторы должны быть хешируемыми, чтобы их можно было использовать в `set` и как ключи `dict`. Хеш должен быть согласован с `__eq__`: если `v1 == v2`, то `hash(v1) == hash(v2)`.

Важно:

- В методах `__add__` и `__mul__` используйте проверку типов и возвращайте `NotImplemented`, если тип не подходит.
- В Python при переопределении `__eq__` атрибут `__hash__` автоматически становится `None`, и объект перестаёт быть хешируемым. Поэтому `__hash__` нужно задать явно.

Задача 4. Наследование: «Сотрудники и Менеджеры»

Есть базовый класс `Employee`:

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def calculate_bonus(self):
        return self.base_salary * 0.10  # 10% от зарплаты
```

Создайте класс `Manager`, который **наследуется** от `Employee`.

1. Конструктор `Manager` принимает `name`, `base_salary` и `team_size` (размер команды). Используйте `super()` для инициализации базовых атрибутов.
2. Переопределите метод `calculate_bonus()`: бонус менеджера составляет 10% от базовой зарплаты **плюс** 5% от базовой зарплаты за каждого члена команды (`team_size`).

Скелет кода

```python
import math

# === ЗАДАЧА 1 ===
class Wallet:
    def __init__(self, initial_balance):
        pass # Ваш код
    def deposit(self, amount):
        pass # Ваш код
    def withdraw(self, amount):
        pass # Ваш код
    def get_balance(self):
        pass # Ваш код


# === ЗАДАЧА 2 ===
class Triangle:
    def __init__(self, a, b, c):
        pass # Ваш код
    def perimeter(self):
        pass # Ваш код

class Circle:
    def __init__(self, r):
        pass # Ваш код
    def perimeter(self):
        pass # Ваш код

def calculate_total_perimeter(shapes):
```

```
    pass # Ваш код


# === ЗАДАЧА 3 ===
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        pass # Ваш код
    def __mul__(self, other):
        pass # Ваш код
    def __eq__(self, other):
        pass # Ваш код
    def __hash__(self):
        pass # Ваш код
    def __str__(self):
        pass # Ваш код


# === ЗАДАЧА 4 ===
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary
    def calculate_bonus(self):
        return self.base_salary * 0.10
```

```
class Manager(Employee):
    pass # Ваш код
```

Тесты для автопроверки

Этот код можно запускать на сервере LMS или локально. Тесты проверяют не только "счастливый путь", но и граничные случаи (ошибки типов, негативные сценарии).

```
import pytest
import math

# --- Тесты Задачи 1 (Инкапсуляция) ---
def test_wallet_initialization():
    w = Wallet(1000)
    assert w.get_balance() == 1000
    w_neg = Wallet(-500)
    assert w_neg.get_balance() == 0, "Баланс не
может быть отрицательным при создании"

def test_wallet_operations():
    w = Wallet(100)
    assert w.deposit(50) is True
    assert w.get_balance() == 150
    assert w.deposit(-10) is False
    assert w.get_balance() == 150
```

```python
    assert w.withdraw(50) is True
    assert w.get_balance() == 100
    assert w.withdraw(200) is False
    assert w.withdraw(-10) is False
    assert w.get_balance() == 100

# --- Тесты Задачи 2 (Полиморфизм) ---
def test_polymorphism_perimeter():
    t = Triangle(3, 4, 5)
    c = Circle(10)
    shapes = [t, c, Triangle(1, 1, 1)]

    # Ожидаемая сумма: 12 + 62.8318... + 3
    expected = 12 + (2 * math.pi * 10) + 3
    assert
math.isclose(calculate_total_perimeter(shapes),
expected, rel_tol=1e-5)

# --- Тесты Задачи 3 (Dunder-методы) ---
def test_vector_add():
    v1 = Vector2D(1, 2)
    v2 = Vector2D(3, 4)
    v3 = v1 + v2
    assert v3.x == 4 and v3.y == 6
    assert v1.x == 1, "Исходный вектор не
должен изменяться"

def test_vector_mul():
```

```
    v = Vector2D(2, 3)
    v2 = v * 3
    assert v2.x == 6 and v2.y == 9

def test_vector_eq_and_str():
    v1 = Vector2D(5, 10)
    v2 = Vector2D(5, 10)
    assert v1 == v2
    assert str(v1) == "(5, 10)"

def test_vector_not_implemented():
    v = Vector2D(1, 1)
    # Проверяем, что при сложении с
неподходящим типом не падает с TypeError,
    # а корректно обрабатывается (или падает с
правильным TypeError от Python)
    with pytest.raises(TypeError):
        _ = v + "string"

# --- Тесты Задачи 4 (Наследование) ---
def test_manager_inheritance():
    m = Manager("Alice", 50000, 5)
    assert m.name == "Alice"
    assert m.base_salary == 50000
    assert hasattr(m, 'team_size') and
m.team_size == 5

    # Бонус: 10% от 50000 (5000) + 5% от 50000
```

```
за каждого из 5 членов (12500) = 17500
    assert m.calculate_bonus() == 17500.0

    # Проверка, что используется super()
(косвенно, через наличие атрибутов родителя)
    assert isinstance(m, Employee)
```